“SMART RECIPE”

Project Report submitted for the partial fulfilment of grade for the course

Data Structures and Algorithms – 3 (25CS2103E)

in

DEPARTMENT OF COMPUTER SCIENCE ENGINEERING

Academic Year 2026–2027

by

P. Sai Ashwika 2520030224

Chegireddy Thanvi 2520030214

G. Keerthi 2520030416

Team Number: 22 | Section: 9

Under the Guidance of

Dr. S. Vinay Kumar

Associate Professor, Department of Computer Science and Engineering



Koneru Lakshmaiah Education Foundation

(Deemed to be University estd. u/s. 3 of the UGC Act, 1956)
Off-Campus: Bachupally-Gandimaisamma Road, Bowrampet, Hyderabad, Telangana - 500 043.
Phone No: 7815926816, www.klh.edu.in

Koneru Lakshmaiah Education Foundation
(Deemed to be University estd. u/s. 3 of the UGC Act, 1956)
Off-Campus: Bachupally-Gandimaisamma Road, Bowrampet, Hyderabad, Telangana - 500 043.
Phone No: 7815926816, www.klh.edu.in

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

Declaration

We hereby declare that the project report entitled "SMART RECIPE: A Data Structures and Algorithms Based Recipe Recommendation System" is a record of the work carried out by P. Sai Ashwika (2520030224), Chegireddy Thanvi (2520030214), and G. Keerthi (2520030416), in partial fulfilment of the requirements for the course Data Structures and Algorithms – 3 (25CS2103E) during the academic year 2026–2027.

We declare that this report represents our understanding and work on the project and that any external resources used are acknowledged in the references.

P. Sai Ashwika – 2520030224

Chegireddy Thanvi– 2520030214

G. Keerthi – 2520030416

Koneru Lakshmaiah Education Foundation
(Deemed to be University estd. u/s. 3 of the UGC Act, 1956)
Off-Campus: Bachupally-Gandimaisamma Road, Bowrampet, Hyderabad, Telangana - 500 043.
Phone No: 7815926816, www.klh.edu.in

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

CERTIFICATE

This is to certify that the mini-project report entitled **"SMART RECIPE**" is a bonafide work done and submitted by **P Sai Ashwika - 2520030224, Chegireddy Thanvi - 2520030214, G.Keerthi - 2520030416** in partial fulfillment of the requirements for the award of the degree of **BACHELOR OF TECHNOLOGY** in Department of Computer Science Engineering, during the academic year **2026-27.**

Signature of the Guide **Signature of the HOD**

(Dr. S. Vinay Kumar)

Signature of the External Examiner

ACKNOWLEDGEMENT

We express our sincere gratitude to our project guide, Dr. S. Vinay Kumar, Associate Professor, Department of Computer Science and Engineering, for his guidance, encouragement, and valuable suggestions throughout the development of our Smart Recipe project.

We also thank Dr. P. Venkateshwara Rao, Associate Professor and Head of the Department of Computer Science and Engineering, for providing the academic environment and support required to complete this project.

Finally, we thank our faculty members, classmates, and everyone who supported us directly or indirectly. This project helped us understand how data structures and algorithms can be applied to a practical everyday problem.

P. Sai Ashwika – 2520030224

Chegireddy Thanvi– 2520030214

G. Keerthi – 2520030416

ABSTRACT

Smart Recipe is a Data Structures and Algorithms based application designed to help users discover recipes using ingredients that are already available at home. The user can enter available ingredients into a pantry list, and the system searches its recipe collection to identify suitable dishes. When some ingredients are missing, the application can suggest alternative ingredients where suitable information is available.

The project focuses on organizing recipe information and retrieving relevant results efficiently. Data structures such as arrays or lists, linked lists, hash maps, and trees can support ingredient storage, recipe organization, searching, and retrieval. Searching and sorting techniques can be used to identify matching recipes and present useful results. The application can also provide recipe details, substitutions, surprise recipe selection, and a favourites list.

The main goal of Smart Recipe is to save users' time, make better use of ingredients already available at home, and help reduce food wastage. The project demonstrates how core DSA concepts can be applied to solve a common real-world problem through a simple and convenient recipe recommendation system.

TABLE OF CONTENTS

Declaration

Certificate

Acknowledgement

Abstract

List of Figures

1. Introduction

1.1 Overview

1.2 Problem Statement

1.3 Scope of the Project

1.4 Objectives

2. System Requirements

2.1 Functional Requirements

2.2 Non-Functional Requirements

2.3 Hardware and Software Requirements

3. System Design

3.1 System Architecture

3.2 Data Flow

3.3 Data Structures and Algorithms

4. Implementation

4.1 Pantry Management Module

4.2 Recipe Search and Matching

4.3 Recipe Details and Substitutions

4.4 Surprise Recipe and Favourites

5. Results and Discussion

6. Challenges and Limitations

6.1 Challenges

6.2 Solutions

6.3 Limitations

7. Conclusion and Future Work

7.1 Conclusion

7.2 Future Work

7.3 Skills Learned

References

LIST OF FIGURES

Figure 3.1 High-Level Architecture of Smart Recipe

Figure 3.2 Workflow of Ingredient-Based Recipe Search

1. INTRODUCTION

1.1 Overview

Food preparation often begins with checking which ingredients are available at home and deciding what can be cooked with them. People may spend time searching several websites or applications to find recipes that match their ingredients. They may also overlook ingredients in their kitchen, which can lead to unnecessary purchases or food waste.

Smart Recipe provides a simple way to search for recipes based on available ingredients. Users maintain a pantry list, request matching recipes, and view details about selected dishes. If the system has substitution information, it can also suggest alternatives for missing ingredients. A favourites feature helps users keep track of recipes they may want to prepare again.

The project is developed as a practical application of Data Structures and Algorithms – 3. It focuses on representing ingredient and recipe information, searching for matches, and presenting results in a useful form.

1.2 Problem Statement

Finding a suitable recipe from the ingredients available at home can be time-consuming. General recipe searches may return dishes that require many ingredients the user does not have. Users may also have difficulty remembering saved recipes or identifying substitutes for missing items. The problem is to design a simple system that accepts available ingredients, searches a recipe collection, and presents relevant recipe suggestions efficiently.

1.3 Scope of the Project

The project covers ingredient entry and pantry management, recipe matching, recipe-detail display, ingredient substitutions where available, random or surprise recipe selection, and favourites management. The system is intended as an educational DSA project and can be extended later with a larger recipe database, user accounts, nutritional information, and a graphical web interface.

1.4 Objectives

- Allow users to add, view, and clear ingredients in a pantry list.
- Find recipes that match the ingredients entered by the user.

- Show recipe details and available ingredient alternatives.
- Provide a surprise-me option to help users discover a recipe.
- Allow users to add or remove recipes from a favourites list.
- Demonstrate the practical use of data structures and searching techniques.
- Help reduce time spent deciding what to cook and encourage better ingredient use.

2. SYSTEM REQUIREMENTS

2.1 Functional Requirements

- Ingredient management: users can add ingredients and view the current pantry.
- Recipe search: the system compares pantry ingredients with recipe requirements and returns matching or suitable recipes.
- Recipe details: users can view the information stored for a selected recipe.
- Substitutions: the system displays alternatives when substitution data is available.
- Surprise selection: the system selects a recipe for users who do not want to choose manually.
- Favourites: users can add or remove a recipe using its recipe ID and view saved favourites.
- Pantry reset: users can clear the current ingredient list.
- Exit: users can close the application safely.

2.2 Non-Functional Requirements

- Usability: menu options and messages should be simple and understandable.
- Performance: searches should return results without unnecessary processing.
- Reliability: ingredient and recipe operations should produce consistent results.
- Maintainability: recipe data and program modules should be easy to update.
- Portability: the program should run in a suitable Java environment.

2.3 Hardware and Software Requirements

Software: Java Development Kit (JDK), a Java-compatible IDE or terminal, and a recipe data collection. The implementation can be developed and tested using an editor such as Visual Studio Code.

Hardware: a standard laptop or desktop computer capable of running the Java runtime environment. No specialised hardware is required.

3. SYSTEM DESIGN

3.1 High-Level Architecture

The system can be understood as four connected parts: the user interface, pantry manager, recipe processing module, and recipe data store. The user selects a menu option and supplies ingredients. The pantry manager stores the current list. The recipe processing module compares the pantry with stored recipe requirements and prepares the results. The user can then view recipe details, substitutions, or manage favourites.

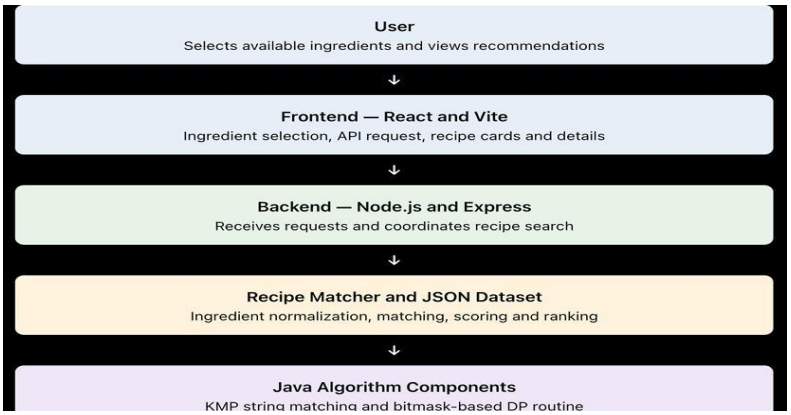


Figure 3.1: High-level architecture of Smart Recipe

3.2 Data Flow

1. The user chooses an operation from the application menu.
2. The user adds ingredients to the pantry or selects an existing pantry operation.
3. For a search, the system reads the pantry ingredients and compares them with the recipe requirements.
4. The system returns suitable matches, which the user can inspect.
5. The user can open recipe details, check available substitutions, save a favourite, or request a surprise recipe.
6. The application updates the relevant list and waits for the next operation.

3.3 Data Structures and Algorithms

Arrays / Lists

Arrays or lists can store ingredients and recipe collections. They provide a straightforward way to iterate through items and compare ingredient names.

Linked Lists

A linked list can be used when the application needs a collection that changes as ingredients or favourite recipes are added and removed. The exact choice depends on the final implementation.

Hash Maps

A hash map can associate a recipe ID or ingredient name with its corresponding information. This can make direct lookups efficient on average.

Trees

A tree can organise recipe information hierarchically, for example by category or other properties, if the project requires structured browsing.

Searching and Sorting

Searching compares the ingredients available in the pantry with the ingredients required by each recipe. Sorting can arrange the results according to a chosen criterion, such as the number of matching ingredients. The specific algorithm and complexity depend on the implementation.

Recipe ID

A recipe ID is a unique identifier assigned to a recipe. It allows the user to select a specific recipe for viewing details or changing its favourite status without having to type the full recipe name.

4. IMPLEMENTATION

4.1 Pantry Management Module

This module accepts one or more ingredients from the user and stores them in the pantry collection. The user can display the current pantry to check which ingredients have been entered. A clear-pantry operation removes the current pantry entries so the user can start a new search.

4.2 Recipe Search and Matching

When the user selects the recipe-search option, the program checks the available ingredients against the ingredients listed for each recipe. Recipes with suitable matches are displayed. A matching approach should account for ingredient names consistently, including case and spacing, so that small text differences do not cause avoidable mismatches.

4.3 Recipe Details and Substitutions

After choosing a recipe, the user can view the stored details. Where the recipe data includes alternatives, the application can suggest substitutions for missing ingredients. Substitution suggestions depend on the information present in the recipe collection and should not be treated as guaranteed equivalents for every recipe.

4.4 Surprise Recipe and Favourites

The surprise-me option selects a recipe from the available collection, helping users discover something without manually browsing every result. If weighted selection is implemented, recipes can be assigned different selection weights. The favourites module lets users add or remove a recipe using its ID and view the list of favourites.

4.5 Menu Options

- 1 – Add ingredient(s) to pantry
- 2 – Show current pantry
- 3 – Find matching recipes
- 4 – Surprise me
- 5 – View recipe details and substitutions
- 6 – Add or remove a favourite by recipe ID
- 7 – Show favourites
- 8 – Clear pantry
- 0 – Exit

The menu provides a simple, step-by-step way to use the core features. Each option should validate user input and display a helpful message when an ingredient or recipe ID is not found.

5. RESULTS AND DISCUSSION

The expected result is an application that accepts a list of ingredients and returns recipe suggestions based on the available recipe data. Users can inspect the pantry, search for matching recipes, view recipe details, and manage favourites. The surprise-selection and substitution features make the system more useful for everyday meal planning.

The project demonstrates how data structures support storage and organisation while algorithms support searching and selection. Final performance depends on the number of recipes, the structure chosen for the data, and how matching is implemented. Screenshots of the running application and sample inputs/outputs should be added here after testing.

Suggested Test Cases

- Add several ingredients and confirm that they appear in the pantry.
- Search with ingredients that match a stored recipe and verify that the recipe is shown.
- Search with ingredients that do not match any recipe and verify that a suitable no-results message appears.
- Open a valid recipe ID and check the displayed details.
- Enter an invalid recipe ID and verify that the program handles it without crashing.
- Add a recipe to favourites, view favourites, and then remove that recipe.
- Clear the pantry and confirm that the pantry is empty.
- Use the surprise-me option more than once and confirm that a valid recipe is selected.

6. CHALLENGES AND LIMITATIONS

6.1 Challenges

- Matching ingredient names when users enter different capitalisation or spacing.
- Finding recipes when only some of their required ingredients are available.
- Keeping pantry data, recipe details, and favourites organised.
- Handling invalid menu choices or unknown recipe IDs.
- Balancing simple program structure with efficient searching.

6.2 Solutions Applied

- Use consistent ingredient-name formatting before comparison.
- Keep recipe data in a consistent structure with clear ingredient lists and IDs.
- Separate pantry operations, recipe matching, and favourites operations into logical modules.
- Validate user input and show understandable error messages.
- Test the program using valid, invalid, empty, and partial-match inputs.

6.3 Limitations of the System

- Recommendations are limited by the size and accuracy of the recipe collection.
- Substitutions can only be suggested when alternatives are recorded in the data.
- The system may not account for allergies, dietary restrictions, nutrition, cooking equipment, or preparation time unless those details are included.
- Results depend on the matching logic and may not reflect personal taste.
- Live online recipe updates or user-specific recommendations are not included unless separately integrated.

7. CONCLUSION AND FUTURE WORK

7.1 Conclusion

Smart Recipe applies Data Structures and Algorithms concepts to a common everyday task: deciding what to cook with ingredients already available at home. By combining pantry management, recipe matching, recipe details, substitutions, surprise selection, and favourites, the project provides a structured approach to recipe discovery. It also demonstrates the importance of organising data carefully and choosing appropriate search techniques.

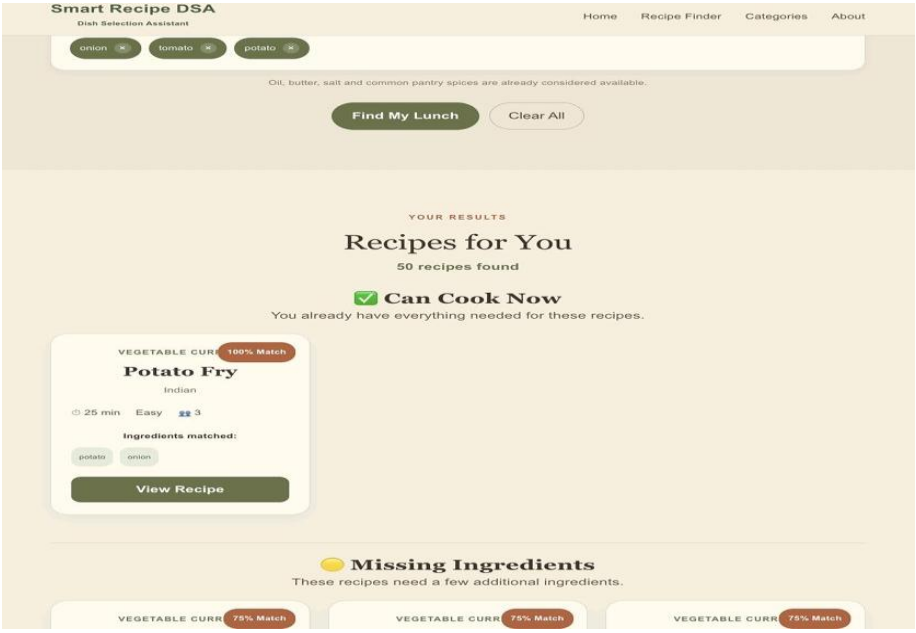
7.2 Future Work

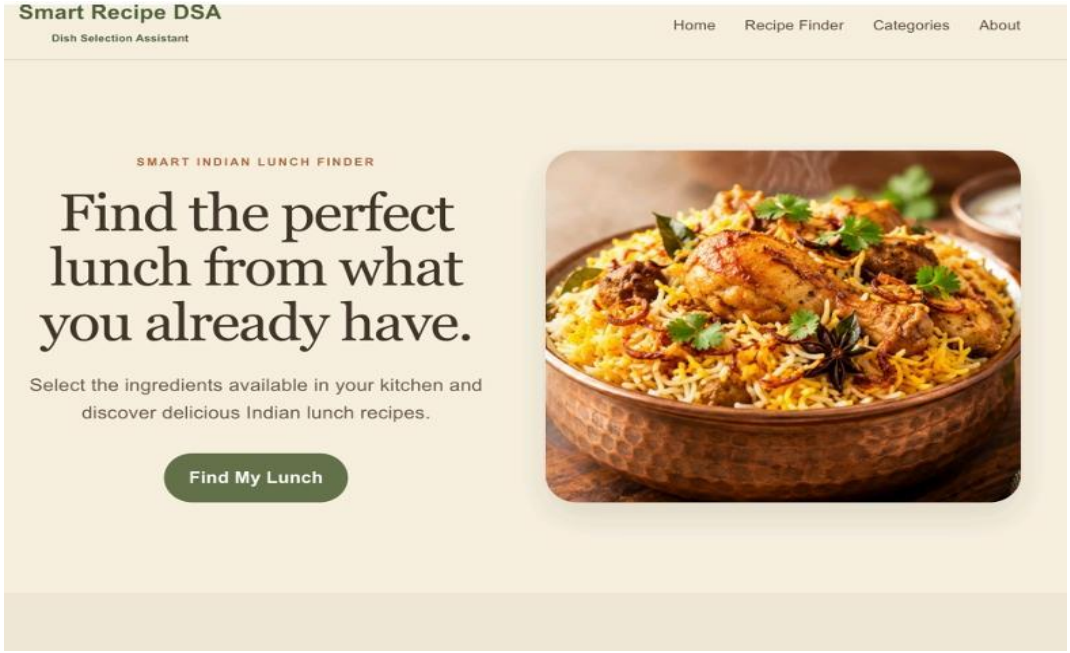
- Develop a graphical web interface for easier use on computers and mobile devices.
- Expand the recipe database with more cuisines and preparation methods.
- Add filters for vegetarian or non-vegetarian dishes, cooking time, difficulty, and dietary preferences.
- Include allergy warnings and more detailed nutritional information.
- Improve ranking by considering the number of missing ingredients and the user's favourites.
- Add user accounts and persistent storage so pantry items and favourites can be saved between sessions.
- Integrate a trusted recipe API, if online data is required and permitted.

7.3 Skills Learned During Development

- Applying data structures to store and organise related information.
- Designing search logic and comparing alternative approaches.

- Breaking a problem into smaller modules.
- Testing program behaviour with different inputs.
- Collaborating as a team and documenting a software project.





REFERENCES

[1] Course material for Data Structures and Algorithms – 3 (25CS2103E), academic year 2026–2027.[2] Oracle, “Java Documentation,” https://docs.oracle.com/en/java/ (accessed October 2026).

[3] Project abstract, “Smart Recipe,” provided by the project team.

[4] Any recipe dataset, tutorials, or external APIs used during implementation should be listed here with their exact titles and URLs.